

Partition-Tolerant Federation Across Emulated ISP Islands

Anonymous submission

Abstract—An Internet shutdown is rarely a clean power switch. Transit to the outside world can be withdrawn while domestic networks keep carrying packets among themselves, leaving reachable fragments that recent measurement work calls connectivity islands. We ask a deliberately narrow question: when some domestic IP paths survive, can independently operated forum servers keep accepting signed posts and keep reconciling them across whatever graph remains? Our prototype combines four mechanisms that are individually familiar and, to our knowledge, have not been composed for this operating regime: canonical protobuf envelopes admitted as the exact bytes their author signed, operator receipts bound to those bytes, caller-initiated synchronisation that survives asymmetric reachability, and an explicitly trusted dual-homed relay between islands. On a four-node testbed whose shared exchange segment is withdrawn mid-run, a dependent certificate, community, and post crossed the severed boundary in 0.5, 1.2, and 2.1 seconds—within a factor of two of the same chain measured while the exchange was intact. An eight-node chain delivered 200 of 200 envelopes seven hops with a 4.125s median and a 6.115s 99th percentile, and per-hop cost tracked a queueing model to within 89ms across a fourfold change in the drain interval. TypeScript, Rust, and Python implementations agreed on all 16 canonical vectors. A signature-invalid flood produced no database writes, though a rate limiter shed 90% of it before the pipeline was reached, which we report rather than headline. Against a median gzipped ActivityPub activity our envelopes are 4.4–6.9× smaller, at the cost of Fediverse interoperability and JSON-LD extensibility. These results establish partition tolerance under surviving IP paths on a single-host testbed. They establish nothing about total blackouts, real inter-provider deployment, or resourced denial of service.

Index Terms—federated systems, Internet shutdowns, network partitions, signed objects, store-and-forward

I. INTRODUCTION

Treating an Internet shutdown as a binary event obscures what operators actually do. Disruptions arrive as throttling, destination blocking, protocol filtering, or the withdrawal of international transit, and they frequently leave domestic reachability partly intact [1], [2]. Baltra *et al.* formalised the resulting structures, naming networks severed from the Internet core *islands* and persistent partial reachability *peninsulas* [3]. Their contribution is measurement: they detect and name the condition. Ours is a system that has to keep working inside it. The question we pose is correspondingly narrow. If domestic IP paths survive a transit cut, can independent forum servers continue to admit signed posts and reconcile them across the surviving graph?

Adjacent systems answer adjacent questions. Circumvention tools such as Telex and Snowflake route a user to an external service through cooperating infrastructure or ephemeral prox-

ies [4], [5]; both presume that some path out of the country exists, which is precisely what a transit cut removes. Device-to-device systems—Anix, Rangzen, Moby, Twimight—abandon the packet network entirely and carry messages on Bluetooth or Wi-Fi Direct encounters [6]–[9], which is the right approach for a total blackout and costs latency, bandwidth, and battery to take. Neither family specifies how independently operated *domestic servers* should validate, store, and relay public forum objects while partial IP connectivity persists. That gap is what this paper occupies.

Our design rests on a single discipline: an author signs one exact byte sequence, and every server that later handles those bytes verifies them unchanged. No relay may normalise, repair, or re-encode an object on its way through. Acceptance produces a receipt signed over the same bytes, so an operator’s acknowledgement is bound to what was actually admitted rather than to an identifier the operator chose. Synchronisation is always initiated by the caller, so a node with no reachable inbound address still both publishes and recovers. When two islands cannot reach each other at all, an explicitly trusted dual-homed server validates, stores, and relays between them, running the same admission code as every other node rather than forwarding packets.

We claim four contributions:

- a byte-preserving admission path applied identically to locally submitted and federated objects, which removes the failure mode in which a relay repairs a malformed object into a verifying one;
- caller-initiated delivery, streaming, and backfill, giving bidirectional federation to nodes that can open connections but not accept them;
- a quota-bounded relay policy that revalidates every object it forwards and excludes the peer it arrived from, so trust governs rate rather than validity; and
- measurements from a testbed in which the inter-island path is withdrawn mid-run, together with cross-language byte-agreement vectors, a hop-scaling chain, a signature-invalid flood, and a compression-aware ActivityPub size comparison.

We claim *partition tolerance under surviving IP paths* and nothing broader. The design cannot manufacture a route through a total communication blackout, cannot conceal a visible relay from the network operator hosting it, and cannot compel an operator to accept a valid post.

II. BACKGROUND AND RESEARCH GAP

Federated and signed systems. ActivityPub delivers social activities to server inboxes [10]; Matrix and the AT Protocol federate rooms and replicate repositories [11], [12]; Nostr and Secure Scuttlebutt replicate author-signed content [13], [14]. Each supplies a mechanism we reuse. None assumes that the server graph itself may fragment: reachable peer addresses remain a background assumption, and no member of the family combines exact-byte admission, operator receipts, outbound-only recovery, and an explicit two-island relay policy in the arrangement evaluated here.

Blackout communication. Anix, Rangzen, Moby, and Twimight move data through physical proximity [6]–[9]; Dolphin smuggles data through a cellular voice channel when packet service is denied [15]. These designs trade latency, throughput, and energy for independence from any domestic server path. Ours makes the opposite trade and inherits the mirror-image weakness: it is fast while servers remain mutually reachable and it stops entirely when every IP path is gone. The two families are complements, not competitors, and we make no throughput or latency comparison against them because the workloads are not commensurable.

Accountability. Certificate Transparency established append-only logs, signed tree heads, and inclusion proofs as a way of making certain misbehaviour visible after the fact [16], [17]; PeerReview and CoSi pursue stronger notions through witnesses and cosigning [18], [19]. Our receipts are a deliberately weaker instrument in that lineage: a receipt proves that one operator acknowledged one specific byte string at one moment, not that the operator accepts all valid objects, forwards them promptly, or refrains from silently declining the next one.

The gap sits between normally routed federation, which assumes the server graph holds, and device-to-device networks, which assume it has vanished. We assume instead that servers stay powered and at least one domestic path survives, and the objective under that assumption is to admit an authored object once, preserve its signed bytes without alteration, and let every reachable replica reach the same verdict on its validity.

III. SYSTEM DESIGN

A. Architecture

A local client and a remote peer converge on the same ingress path; nothing about the transport that carried an object changes how it is judged. Admission inspects the submitted bytes before any persistent write occurs. After the transaction commits, the server returns a receipt and enqueues federation work in a durable outbox, which a scheduler drains grouped by destination, source uplink, and traffic class. A bridge is an ordinary server running that same stack, dual-homed onto two networks. It is not a transparent packet forwarder, and the distinction matters: everything it relays, it has first independently validated and stored under its own key.

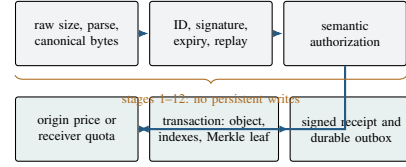


Fig. 1. Admission pipeline. State changes begin only after the cheap byte and validity checks have all succeeded.

B. Signed objects and admission

An envelope carries a domain tag, an object type, a payload, a public key, a nonce, and an expiry. Its identifier is the SHA-256 digest of a specified canonical byte string, and an Ed25519 signature covers that same string [20]. Because protobuf does not guarantee a unique serialisation [21], we constrain a profile on top of it: ascending field order, minimal variants, no unknown fields retained, and—the operative rule—a decode followed by a re-encode that must reproduce the received bytes exactly. An object that fails that comparison is rejected rather than repaired.

Strict-decode-then-verify-what-arrived is long-established practice in DER, JWS, and CT’s leaf format; what it buys here is specific to relaying. A federation node that decoded a peer’s bytes and re-encoded them before verification would silently normalise a malformed object into a well-formed one and then find that it verifies. Under our rule the receiver never normalises attacker-controlled bytes before checking the signature, so no relay can launder an object into validity.

Fig. 1 summarises the admission path. Stages 1 through 12 cover size, parsing, canonicity, domain separation, identifier derivation, signature, expiry, replay, and authorisation, and none of them writes to persistent storage—an invalid flood therefore cannot amplify into write load, a property we measure in §V. Later stages charge origin pricing or a receiver quota, commit the raw object together with its derived indexes in a single transaction, append its hash to a Merkle log, and only then enqueue federation. A uniqueness constraint in the database, rather than a read-then-write check, makes exact retries idempotent under concurrency.

On acceptance the server signs a receipt binding the object identifier, its leaf position, the resulting tree head, and an inclusion proof, so the acknowledgement commits to the bytes admitted rather than to a label the operator is free to change afterwards. Peers additionally exchange signed tree heads. Two heads of equal size with differing roots are an unambiguous conflict; a larger head is recorded as pending until a consistency proof can be fetched. We are explicit about the reach of this rule. It is stale-head hygiene that eliminates a class of self-inflicted false positives—and it does catch outright tree shrinkage—but an adversary who rewrites history while keeping the tree non-decreasing passes both clauses undetected, and equivocation between two peers cannot be caught by any purely local comparison. Detecting either requires consistency proofs and cross-observer exchange, which the partition we are designing for is precisely what obstructs.

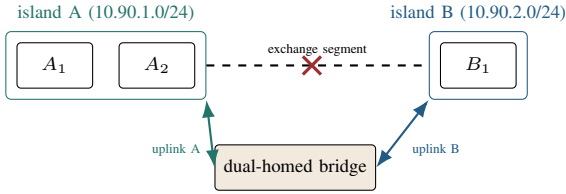


Fig. 2. Route-cut topology. Four application nodes span two isolated network segments joined by a shared exchange, which the experiment withdraws mid-run. The bridge is an application server, not an IP forwarder.

C. Partition-aware federation

Three remote procedure calls, all initiated by the caller, cover ordinary delivery and recovery. `DELIVER` pushes a batch, `STREAMACTIVITIES` resumes from a durable cursor, and `BACKFILL` requests objects a node knows it is missing. Because none of the three requires the calling node to accept an inbound connection, a server behind carrier-grade network address translation participates fully in both directions over connections it opens itself. This is reachability asymmetry, not discovery: at least one peer address must already be known.

The scheduler drains independent per-peer queues concurrently under a bounded lease, with deadlines, exponential backoff, and idempotency keys. That structure exists because its absence was a real defect, described in §V. One caveat is worth stating in the design rather than buried in limitations: the deadline bounds how long a drain pass waits on a peer, not how long the underlying transport keeps trying, because the sender takes no cancellation token. A stuck call can therefore hold resources past its deadline even though it no longer blocks anyone.

Fig. 2 shows the route-cut topology. Two ordinary nodes sit in island A and one in island B; a shared exchange segment connects them, standing in for an Internet exchange point. The bridge holds one address on each island network. Once the exchange is withdrawn, both islands must already have configured trust in the bridge—trust that governs the rate and priority a peer’s traffic receives, never whether its objects are valid. The bridge validates and stores every object it handles, bounds volume per uplink pair and per traffic class, and never relays an object back to the peer it arrived from.

Four implementation rules carry most of the replication safety. The transport hands ingress an opaque byte string and has no ability to decode and re-encode it. Every peer, the bridge included, passes the complete validity check. Database uniqueness, not application logic, absorbs the duplicate that inevitably arrives when a live stream and a backfill both carry the same object. Finally, fan-out excludes the originating peer, which avoids a redundant round trip that content deduplication would eventually terminate but not for free.

IV. METHODOLOGY

We evaluate five questions: whether independent encoders agree on the canonical form, whether objects cross an enforced route cut, how delivery latency scales with hop count, whether

malformed requests produce storage writes, and how the representation compares in size with ActivityPub. All experiments run on one physical host; §VI treats the consequences of that at length.

Correctness tests. A shared corpus of 16 positive and adversarial byte vectors is executed independently by the TypeScript, Rust, and Python implementations, each of which must agree byte-for-byte on encoding and identifier derivation and must reject every negative vector. A further 85 focused tests cover ingress (29), the outbox (3), federation (31), and simulated ISP behaviour (22). These are regression tests written by the system’s authors, not an independent security audit.

Route-cut experiment. Four application nodes, two databases, and two caches are distributed across three isolated virtual network segments as in Fig. 2. Containers supply reproducible layer-2 isolation and let the harness withdraw the only configured inter-island path mid-run; nothing in the design depends on that packaging. The gate reads kernel TCP state from `/proc/net/tcp` inside the bridge to confirm which local source address each established federation connection actually left from, withdraws the exchange segment, verifies that direct probes between islands now fail and that island B does not already hold the content, submits a dependent certificate, community, and post, and polls for each exact content identifier. Nineteen assertions cover topology, source binding, relay behaviour, quota reservation, and uplink failover. Reported latencies include the application’s own polling interval and are therefore upper bounds.

Hop-scaling chain. Eight nodes are peered into a line, each with its own database, so that a node i hops from the origin is reachable by exactly one path of exactly that length. Hop count is the independent variable, which a mesh would destroy, since in a mesh every node is one hop away and only fan-out width varies. Latency is computed by subtracting the `received_at_ms` value each node writes inside the same transaction as its projection, so no polling observer sits in the measurement path; this is sound only because all nodes share the host clock, which a real deployment would have to replace with NTP discipline and an error bar. We inject 200 valid envelopes at hop 0 at a paced rate and record arrival at hop 7. The eight nodes share a single database process with one database each, which introduces contention and inflates the reported figures—the conservative direction in which to be wrong.

Signature-invalid flood. Random bytes are refused within microseconds at the parse stage and measure nothing interesting. The tool therefore lifts a genuine envelope from the node’s own log and mutates a single byte inside the signed region, which invalidates the signature and changes the content identifier, so every request is novel and none can be short-circuited by deduplication. We issue 3,000 such requests at concurrency 16 and record completion rate, rejection stage, the fraction reaching signature verification, and—via the database’s collection-level profiler—the resulting writes. The ordinary rate limiter stays enabled, so this is not a saturation

benchmark.

Encoding comparison. We collected 80 real Create/Note activities from the public outboxes of two stock ActivityPub deployments. Because a collection hoists @context to the collection while an activity POSTed to a remote inbox carries its own, measuring the bare outbox item would understate delivery size; the tool refetches a standalone object from the same host and re-attaches exactly those bytes. We compare raw and gzipped activity bytes against equivalent signed envelopes, subtracting authored UTF-8 text so the metric reflects what a protocol designer controls rather than how much a user typed. The comparison is deliberately generous to ActivityPub, excluding HTTP framing, TLS, per-follower fan-out, shared dictionaries, and—most significantly—authentication, since our bytes carry a 64-byte signature verifiable at rest while HTTP Signatures are transport level and vanish with the request.

V. RESULTS

A. Correctness and convergence

Every implementations agreed on all the 16 canonical vectors and rejected all the negative ones as specified. The additional tests also checked invalid signatures, incorrect byte formats, replay attacks, missing permissions, duplicate messages, wrong signing keys, invalid proofs, outdated tree heads, and conflicting logs. Sixteen vectors cannot cover every protobuf’s edge cases. However, they show that those three independently developed implementations follow exactly the same byte-level rules. This is essential for a relay network to work reliably.

The route-cut test passes all 19 checks. The TCP connection check also confirmed that the federation connections used both configured addresses, 10.90.1.30 and 10.90.2.30. But this could only be verified with real network sockets, not an in-process test. When direct communication between the islands was disabled, Island B still received the certificate after 0.5 s, the community after 1.2 s, and the post after 2.1 s. This follows their dependency order through the bridge. For comparison, with the direct connection still available, the same items took 1.5 s, 1.5 s, and 1.9 s. So the bridge worked correctly and was still within about twice the speed of the direct connection. These are results from single test runs, and not statistical averages.

The 2.1 s result was achieved through a specific fix and was not just simply observed. An earlier version passed 18 of 19 tests, but the one successful crossing took 407.6 s, with all outbox records still showing zero delivery attempts. The problem was that peers were contacted sequentially with no time limit. If one peer was unreachable, it blocked delivery to all the others. This includes the bridge that was supposed to bypass the network failure. We were able to fix this by delivering to peers concurrently and adding a time limit to each delivery. This reduced the crossing time from 407.6 s to 2.1 s, which is a 195× improvement.

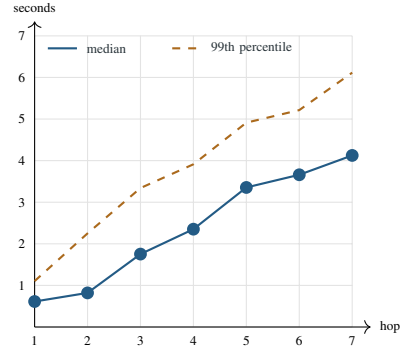


Fig. 3. Measured per-hop latency across the eight-node chain at a 1,000 ms drain interval, from in-transaction timestamps. All 200 envelopes reached hop 7.

B. Latency scaling and admission cost

All 200 envelopes successfully traveled through the eight-node chain to hop 7. The median End-to-end latency was 4.125 s and the 99th percentile 6.115 s. The per hop results which are in Fig. 3 are more useful than the final numbers. Most of the delay comes from the outbox drain interval, not the protocol itself, which if predicted roughly then it’s $drain/2$ plus a fixed term; With a 1,000 ms drain interval, each hop took about 589 ms, including roughly an 89 ms fixed protocol work. With a 250 ms interval, each hop took about 196 ms, implying 71 ms of fixed work. Since this fixed cost stayed around 71–89 ms even when the drain interval changed, it actually represents the protocol’s actual per-hop cost. This includes verifying, projecting, appending, and re-queuing the message. The remaining delay is mainly a scheduling choice that operators can adjust based on bandwidth and battery needs. However, these results should not be treated as Internet performance predictions because the test used a shared database process and did not include real wide-area network delays.

The invalid-signature flood was rejected at 730 requests per second. This was all done with no writes to objects, indexes, logs, or the outbox. The test also confirmed that the profiler was actually running as it recorded six read operations during that test. Of the 3,000 total requests, about 2,700 were blocked by the per-peer rate limiter. So, only around 9% reached signature verification, and then the expensive rejection process, which is the canonical parsing followed by Ed25519 verification. This ran about 66 requests per seconds. The final conclusion is a bit limited but clear enough. Invalid traffic was stopped before causing any writes, with the rate limiter handling most of the attack first. This test does not necessarily show how the system would handle floods from many different peers, valid messages that should cause writes, or long-term parser and database pressure.

Table I compares two encoding formats and their costs. Both tested hosts produced the same 927-byte @context declaration, which implies this is a fixed cost of the standard ActivityPub format, not something specific to our deployment. The declaration alone is more than four times larger than our

entire signed forum post. Compression removes much of the apparent size advantage, so we use compressed ActivityPub numbers for a fair comparison. Gzipped ActivityPub messages were 786–1,400 B with a median of 1,074 B. Our raw signed envelopes were only 155 B, 220 B, and 243 B making ActivityPub about 4.4–6.9 $\times$ larger. Our envelopes do not benefit from compression at all as the 64-byte Ed25519 signature is incompressible and gzip makes all three slightly *larger*. So the honest comparison is gzipped ActivityPub against our raw bytes, not the roughly 20 $\times$ ratio suggested by comparing uncompressed sizes. However, size is the only area where our format is better. It cannot connect to the existing Fediverse, requires generated codecs and unfamiliar tools, does not allow unknown fields, and therefore requires versioned schemas for future changes.

VI. DISCUSSION AND LIMITATIONS

Testbed validity. All experiments were run on a single physical machine using virtual networks in Docker. This is the paper’s main limitation. All nodes shared the same kernel, clock, CPU, and storage. The shared clock made it possible to compare timestamps across nodes, but real deployments do not have this advantage. The virtual networks were also much simpler than real networks, which involve physical routers, BGP changes, NAT, DNS failures, different MTUs, and wireless connections. Similarly, connecting a bridge to two virtual networks is easy, while a real deployment would require separate network links, routing rules, credentials, power, and an operator. Therefore, the route-cut experiment shows that the software can react correctly to a change in network and routing state. It does not demonstrate real geographic latency, independent failure domains, or real-world provider behaviour.

Access and subscriber isolation. The system assumes that nodes on the same network can communicate with each other or reach a shared bridge. However, some Wi-Fi networks and ISPs block direct communication between customers while still allowing them to access the provider’s gateway. If this restriction exists on every available path, nodes cannot connect to each other or reach the bridge. In that situation, federation completely stops, and only local services continue to work. The current prototype has no alternative non-IP method for communication in this situation.

Network adversary. A network operator can do multiple things. They range from find stable endpoints, block IP addresses or SNI values, identify the RPC traffic, track communication timing, isolate a node to pressuring a bridge operator. Source-address binding only chooses which network connection to use; it does not hide the traffic. There is also one important failure case: **protocol whitelisting**. In this situation, normal domestic Internet routing still works, but deep packet inspection allows only approved protocols. Since the federation uses a distinctive, non-standard protocol on a specific port, it could be blocked immediately, regardless of how strong our signatures are. Possible solutions include running federation over normal HTTPS, changing endpoints

regularly, or using pluggable transports. However, none of these solutions has been implemented or tested in this work.

System and application limits. Signatures prove who published something, but they do not prove that the content is truthful or safe. A legitimate key can still be used to publish harmful content or generate a large amount of traffic. Because the cost of admission is paid at the origin, an origin operator can create unlimited free proofs for its own users. Therefore, protection against abuse is only as strong as the weakest origin in the peer group. Per-peer limits are the main protection. Other problems remain open, such as stolen keys, exposed metadata, large-scale moderation, recovering from revoked keys, running out of storage, and coordinated fake identities (Sybil attacks). Transport timeouts only limit how long the system waits; they do not actually stop work that has already started. The system also specifies consistency proofs for growing trees, but these have not yet been implemented. Most importantly, a receipt only proves what the system **accepted and acknowledged**. If an operator simply refuses to accept something, no receipt is created. So, the simplest form of censorship which is silence or refusal to admit content, is impossible for this system to prove.

Evaluation limits. The route-cut results come from single test runs using one configuration. The encoding comparison used 80 activities of one type from two deployments. The system wasn’t tested across multiple physical hosts, real wide-area networks, several days, or actual shutdowns. Host load, results from repeated runs, or confidence intervals weren’t recorded. The resource measurements were taken on x86 computers, so they may not represent how the system would perform on the low-cost single-board computers that communities might actually use. Finally, the comparisons with proximity-based and voice-channel systems involve different types of workloads. They are meant for context, not to rank one system against another.

Ethics. All tests used fake content on isolated private networks, so no real users or sensitive data were involved. However, a real deployment could put bridge operators and users at legal or physical risk because a visible relay can be identified by the network hosting it. Therefore, the system should not be deployed in a hostile environment without first assessing local threats, getting informed consent, minimizing collected data, and reviewing the deployment with the affected community.

VII. CONCLUSION

Server federation can continue working after a route is cut, as long as domestic IP connections are still available. The system uses signed objects, one admission process for both local and federated traffic, caller-controlled recovery, and a trusted relay. Together, these allowed a dependent post to cross the broken connection in 2.1 s, which was within twice the time of the normal direct connection. The problem that caused an earlier 407.6 s delay only became visible when testing with three peers, including one unreachable peer. Tests across different programming languages also showed that they

TABLE I

MEASURED REPRESENTATION SIZE AND DESIGN TRADE-OFFS AGAINST ACTIVITYPUB. THE BYTE SAMPLE IS 80 REAL CREATE/NOTE ACTIVITIES FROM TWO STOCK DEPLOYMENTS. RATIOS COMPARE GZIPPED ACTIVITYPUB AGAINST RAW SIGNED ENVELOPES, WHICH DO NOT COMPRESS.

Dimension	This work	ActivityPub comparison	Cost or boundary
Measured bytes	Raw signed envelopes: check-in 155 B, forum post 220 B, broadcast 243 B; gzip enlarges all three	Delivered JSON-LD median 4,216 B, gzipped median 1,074 B (range 786–1,400 B); @context alone a fixed 927 B	Ratio 4.4–6.9×; representation only—HTTP, TLS, fan-out, shared dictionaries, and AP authentication all excluded in AP’s favour
Interoperability	Custom protobuf domains and RPCs	Standard actor, object, and inbox model	No Fediverse compatibility and no existing application ecosystem
Evolution	Unknown fields rejected; versioned profiles required	Flexible JSON-LD vocabulary and open extension	Harder rolling upgrades and more cross-language test surface
Operations	Generated codecs and canonical-byte tooling required	Text inspectable with ordinary web tools	Higher debugging and tooling burden on operators
Integrity profile	One exact byte string per object, content ID, author signature, and receipt	Authentication depends on deployment profile; HTTP Signatures do not survive the request	Unambiguous byte identity, but a new security profile to maintain and audit

produced identical data, while the multi-hop test showed that the per-hop delay closely followed the expected queueing model, with only about 89 ms of additional cost. The compact format is 4.4–6.9× smaller than gzipped ActivityPub but this comes at the cost of interoperability, extensibility, and mature tools. The most important future work is testing on multiple physical hosts and real wide-area networks, adding consistency proofs for growing logs, making transport cancellation-aware, testing floods of validly signed traffic, using multiple independent bridges, hiding the protocol from systems that allow only approved protocols, and providing a non-IP fallback for subscriber isolation and complete network outages.

REFERENCES

- [1] Z. S. Bischof, K. Pitcher, E. Carisimo, A. Meng, R. B. Nunes, R. Padmanabhan, M. E. Roberts, A. C. Snoeren, and A. Dainotti, “Destination unreachable: Characterizing internet outages and shutdowns,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023.
- [2] T. I. Raso, S. Afrin, M. A. Chowdhury, M. Xynou, and E. Yachmeneva, “The longest silence: Internet shutdowns during Bangladesh’s 2024 uprising,” Digitally Right and Open Observatory of Network Interference, 2025, <https://ooni.org/post/2025-bangladesh-report/>.
- [3] G. Baltra, T. Saluja, Y. Pradkin, and J. Heidemann, “Reasoning about internet connectivity,” 2024.
- [4] E. Wustrow, S. Wolchok, I. Goldberg, and J. A. Halderman, “Telex: Anticensorship in the network infrastructure,” in *Proceedings of the 20th USENIX Security Symposium*, 2011.
- [5] C. Bocovich, A. Breault, D. Fifield, Serene, and X. Wang, “Snowflake, a censorship circumvention system using temporary WebRTC proxies,” in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [6] S. Kamali and D. Barradas, “Anix: Anonymous blackout-resistant microblogging with message endorsing,” in *2025 IEEE Symposium on Security and Privacy*, 2025, pp. 1381–1399.
- [7] A. Lerner, G. Fanti, Y. Ben-David, J. Garcia, P. Schmitt, and B. Raghavan, “Rangzen: Anonymously getting the word out in a blackout,” 2016.
- [8] A. Pradeep, H. Javaid, R. Williams, A. Rault, D. Choffnes, S. Le Blond, and B. Ford, “Moby: A blackout-resistant anonymity network for mobile devices,” *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 3, pp. 247–267, 2022.
- [9] T. Hossmann, P. Carta, D. Schatzmann, F. Legendre, P. Gunningberg, and C. Rohner, “Twitter in disaster mode: Security architecture,” in *Proceedings of the First ACM Workshop on Privacy and Security in Mobile Systems*, 2011.
- [10] C. L. Webber, J. Tallon, E. Shepherd, A. Guy, and E. Prodromou, “ActivityPub,” W3C Recommendation, 2018, <https://www.w3.org/TR/activitypub/>.
- [11] The Matrix.org Foundation, “Matrix specification, version 1.11,” 2024, <https://spec.matrix.org/v1.11/>.
- [12] Bluesky Social, “AT Protocol repository specification,” 2026, <https://atproto.com/specs/repository>.
- [13] fiatjaf, “NIP-01: Basic protocol flow description,” 2023, <https://github.com/nostr-protocol/nips/blob/master/01.md>.
- [14] D. Tarr, E. Lavoie, A. Meyer, and C. Tschudin, “Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications,” in *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*, 2019, pp. 1–11.
- [15] P. K. Sharma, R. Sharma, K. Singh, M. Maity, and S. Chakravarty, “Dolphin: A cellular voice based internet shutdown resistance system,” *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 1, pp. 589–607, 2023.
- [16] B. Laurie, A. Langley, and E. Kasper, “Certificate transparency,” Internet Engineering Task Force, Tech. Rep. RFC 6962, 2013.
- [17] B. Laurie, E. Messeri, and R. Stradling, “Certificate transparency version 2.0,” Internet Engineering Task Force, Tech. Rep. RFC 9162, 2021.

- [18] A. Haeberlen, P. Kouznetsov, and P. Druschel, "PeerReview: Practical accountability for distributed systems," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 2007, pp. 175–188.
- [19] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, "Keeping authorities honest or bust with decentralized witness cosigning," in *2016 IEEE Symposium on Security and Privacy*, 2016, pp. 526–545.
- [20] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [21] Google, "Protobuf serialization is not canonical," 2024, <https://protobuf.dev/programming-guides/serialization-not-canonical/>.